

Relatório de Homologação e Análise de Defeitos

1. Identificação do Projeto e da Equipe

- Nome do Sistema: NinjaForge 3D
- Versão / Build testada: Ex: v1.0.0-rc1
- Data da Homologação: 22/09/2026
- Responsáveis (Equipe de QA): Acassio/ Ariel / Fernanda

2. Resumo Executivo dos Testes

- Total de Casos de Teste Planejados: 3
- Casos Executados: 3
- Casos com Sucesso (Passed): 0
- Casos com Falha (Failed): 3
- Total de Defeitos Encontrados: 3
- Status Atual do Sistema: ● NÃO APROVADO]

3. Matriz de Análise e Classificação de Defeitos

Tabela detalhando os defeitos encontrados, suas classificações e o planejamento de reteste/regressão:

ID Defeito	Descrição Resumida do Problema	Severidade	Prioridade	Plano de Reteste	Testes de Regressão Relacionados
DEF-01	API permite cadastro de produto com preço negativo.	Alta	Alta	Enviar produto com preço negativo e verificar se o cadastro é bloqueado	Testar cadastro de produto com preço positivo e verificar se o fluxo válido

ID Defeito	Descrição Resumida do Problema	Severidade	Prioridade	Plano de Reteste	Testes de Regressão Relacionados
					continua funcionando.
DEF-02	API permite cadastro de produto com preço igual a zero.	Alta	Alta	Enviar produto com preço 0 e verificar se o cadastro é bloqueado	Testar cadastro de produto com preço positivo e verificar se o produto é cadastrado corretamente.
DEF-03	API permite cadastro de produto com quantidade negativa.	Alta	Alta	Enviar produto com quantidade negativa e verificar se o cadastro é bloqueado.	Testar cadastro de produto com quantidade positiva e verificar se o cadastro e os demais dados do produto continuam funcionando.

4. Evidências dos Testes e Defeitos

Para cada defeito crítico ou de alta relevância, anexe evidências visuais (prints de tela, logs ou vídeos) que comprovem o comportamento incorreto.

DEF-01 — Preço negativo

Passos para reproduzir:

1. Acessar o endpoint de cadastro de produtos;

2. Enviar uma requisição utilizando o método **POST**;
3. Informar um valor negativo no campo preço;
4. Enviar a requisição.

Dados enviados:

```
{
  "id": 1,
  "nome": "Tarzan",
  "preco": -10.50,
  "quantidade": 2,
  "id_categoria": 1
}
```

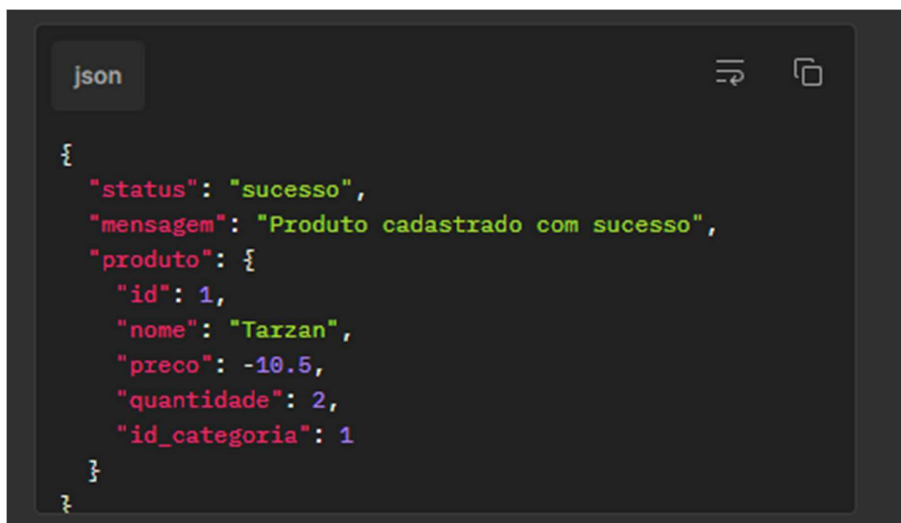
Comportamento Esperado:

O sistema deveria rejeitar o cadastro e retornar uma mensagem informando que o produto não pode ser cadastrado com preço negativo.

Comportamento Observado:

A API retornou **200 OK** e cadastrou o produto mesmo com o preço negativo.

Evidência Visual:



A screenshot of a code editor showing a JSON response. The JSON is formatted with syntax highlighting and includes a tab labeled 'json' at the top left. The response indicates a successful registration of a product.

```
{
  "status": "sucesso",
  "mensagem": "Produto cadastrado com sucesso",
  "produto": {
    "id": 1,
    "nome": "Tarzan",
    "preco": -10.5,
    "quantidade": 2,
    "id_categoria": 1
  }
}
```

DEF-02 - Cadastro com preço igual a zero

Passos para Reproduzir:

1. Acessar o endpoint de cadastro de produtos;

2. Enviar uma requisição utilizando o método **POST**;
3. Informar o valor 0 no campo preço;
4. Enviar a requisição.

Dados enviados:

```
{
  "id": 1,
  "nome": "Tarzan",
  "preco": 0,
  "quantidade": 2,
  "id_categoria": 1
}
```

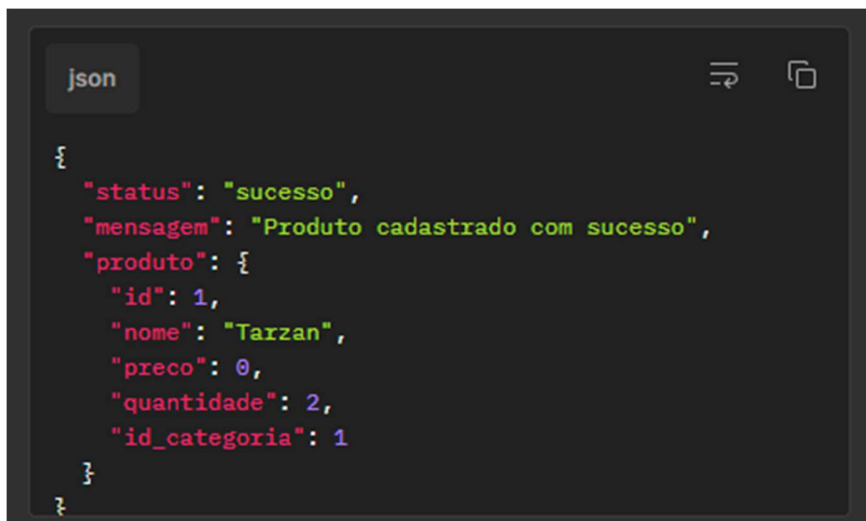
Comportamento Esperado:

O sistema deveria rejeitar o cadastro e informar que o produto não pode ser cadastrado com preço igual a zero.

Comportamento Observado:

A API retornou **200 OK** e cadastrou o produto com preço igual a zero.

Evidência Visual:



A screenshot of a code editor showing a JSON response. The editor has a dark background with a 'json' tab at the top left. The JSON object is as follows:

```
{
  "status": "sucesso",
  "mensagem": "Produto cadastrado com sucesso",
  "produto": {
    "id": 1,
    "nome": "Tarzan",
    "preco": 0,
    "quantidade": 2,
    "id_categoria": 1
  }
}
```

DEF-03 - Cadastro com quantidade negativa

Passos para Reproduzir:

1. Acessar o endpoint de cadastro de produtos;
2. Enviar uma requisição utilizando o método **POST**;

3. Informar um valor negativo no campo quantidade;
4. Enviar a requisição.

Dados enviados:

```
{
  "id": 1,
  "nome": "Tarzan",
  "preco": 10.50,
  "quantidade": -2,
  "id_categoria": 1
}
```

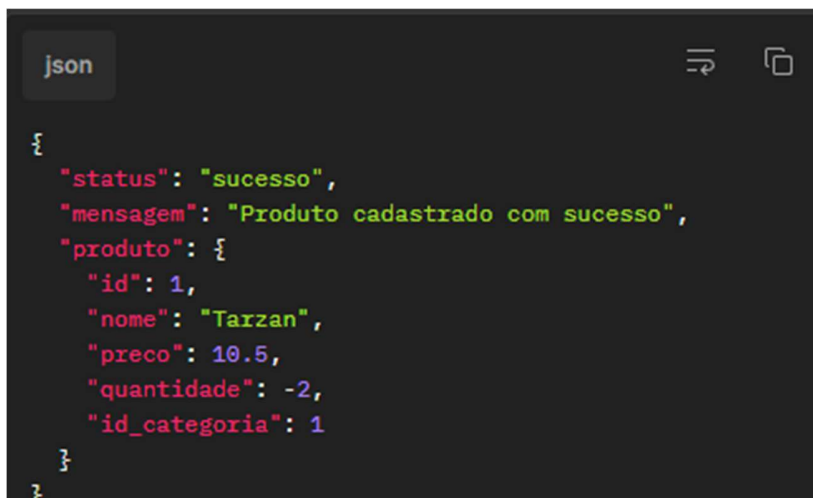
Comportamento Esperado:

O sistema deveria rejeitar o cadastro e informar que o produto não pode ser cadastrado com quantidade negativa.

Comportamento Observado:

A API retornou **200 OK** e cadastrou o produto mesmo com a quantidade negativa.

Evidência Visual:



A screenshot of a code editor showing a JSON response. The response is a success message indicating that the product was registered successfully. The product details are: id: 1, nome: Tarzan, preco: 10.5, quantidade: -2, and id_categoria: 1. The JSON is formatted with syntax highlighting.

```
{
  "status": "sucesso",
  "mensagem": "Produto cadastrado com sucesso",
  "produto": {
    "id": 1,
    "nome": "Tarzan",
    "preco": 10.5,
    "quantidade": -2,
    "id_categoria": 1
  }
}
```

5. Plano de Reteste e Regressão Executado

- **Retestes Realizados:**

- Reexecutar o teste de preço negativo após a correção da API;

- Reexecutar o teste de preço igual a zero;
- Reexecutar o teste de quantidade negativa;
- Confirmar que produtos com dados válidos continuam sendo cadastrados normalmente.

- **Testes de Regressão Realizados:**

- Cadastro de produto com preço positivo;
 - Cadastro de produto com quantidade positiva;
 - Verificação dos dados retornados pela API após um cadastro válido;
 - Verificação de que os campos obrigatórios continuam sendo aceitos corretamente.
-

6. Parecer Final e Conclusão da Homologação

Parecer escolhido: ● **NÃO APROVADO**

Justificativa:

Os testes realizados identificaram três comportamentos incorretos na API de cadastro de produtos. O sistema permite atualmente o cadastro de produtos com preço negativo, preço igual a zero e quantidade negativa, retornando 200 OK e confirmando o cadastro. Esses comportamentos estão em desacordo com as regras esperadas para os dados do produto. Dessa forma, é necessário corrigir as validações da API e realizar novos testes antes da homologação final.